

Experiment No. 6

Title:

Demonstration of Digital Signature Creation and Make use of Digitally Signed Certificate

Aim:

To design and demonstrate a secure digital question paper distribution system using Digital Signatures and Certificate Authority (PKI) where:

- The Exam Server digitally signs the question paper using **Adobe Acrobat Reader**
- The Exam Client verifies authenticity using a trusted CA certificate
- Integrity and authenticity of the document are validated

Capstone Scenario

A University Examination Cell distributes question papers digitally to affiliated colleges.

Security Requirements:

1. Question paper must not be modified.
2. Only authorized Exam Server can sign.
3. Colleges must verify authenticity before printing.
4. Any tampering must be detectable.

Entities:

- University Root CA (Trust Anchor)
- Exam Server (Signer)
- College Exam Client (Verifier)
- Attacker (Tampering attempt)

Backend certificate creation is done using **OpenSSL** on **Kali Linux**.

Theory

1. Digital Signature

A digital signature ensures:

- Integrity (document not modified)
- Authentication (signed by genuine authority)
- Non-repudiation (signer cannot deny)

Process:

1. Document hash is generated using SHA-256.
2. Hash is encrypted using sender's private key.
3. Signature is embedded in PDF.
4. Receiver decrypts hash using public key.
5. Hashes are compared.

2. Public Key Infrastructure (PKI)

PKI consists of:

- Root Certificate Authority (CA)
- Digital Certificates
- Public and Private Keys
- Trust Chain

Trust model:

Root CA → Signs Exam Server Certificate → Signs Question Paper

Verification checks:

- Signature validity
- Certificate issuer
- Trust anchor
- Expiry date
- Document modification

3. X.509 Certificate and Certificate Authority

Algorithm

Step 1: Create Root CA

Generate CA key and certificate.

Step 2: Issue Certificate to Exam Server

Generate server key → Create CSR → CA signs certificate.

Step 3: Convert Certificate to PKCS#12

Required for Adobe signing.

Step 4: Exam Server Signs Question Paper

Use Adobe digital signature tool.

Step 5: Exam Client Verifies

Install CA certificate → Open signed PDF → Validate signature.

Step 6: Tampering Test

Modify PDF → Verify failure.

Procedure (Tasks)

◆ Task 1: Create University Root CA (Kali Linux)

```
openssl genrsa -out university_ca.key 4096
```

```
openssl req -x509 -new -nodes -key university_ca.key -sha256 -days 3650 -out university_ca.crt
```

◆ Task 2: Issue Certificate to Exam Server

```
openssl genrsa -out exam_server.key 2048
```

```
openssl req -new -key exam_server.key -out exam_server.csr
```

```
openssl x509 -req -in exam_server.csr \
```

```
-CA university_ca.crt \
```

```
-CAkey university_ca.key \
```

```
-CAcreateserial \
```

```
-out exam_server.crt \
```

```
-days 365 -sha256
```

◆ Task 3: Convert to Adobe-Compatible Certificate

```
openssl pkcs12 -export \  
-out exam_server.p12 \  
-inkey exam_server.key \  
-in exam_server.crt  
Set password.
```

◆ Task 4: Sign Question Paper using Adobe (Exam Server)

1. Install exam_server.p12 in Adobe
Preferences → Signatures → Digital IDs → Import
2. Open question_paper.pdf
3. Tools → Certificates → Digitally Sign
4. Select certificate
5. Save signed PDF

File generated:

question_paper_signed.pdf

◆ Task 5: Verify at Exam Client

1. Install university_ca.crt in Adobe
Preferences → Trusted Certificates → Import
2. Mark:
✓ Trust for signatures
3. Open signed PDF

Expected Output:

✓ “Signed and all signatures are valid.”

◆ Task 6: Tampering Test

Modify signed PDF.

Open again.

Expected:

✗ “Signature is invalid”

✗ “Document has been altered”

Result

- Root CA successfully created.
- Exam Server certificate issued.
- Question paper digitally signed.
- Client successfully verified signature.
- Tampered document detected.
- PKI trust chain validated.

Conclusion

The experiment successfully demonstrated a secure digital question paper distribution system using PKI and digital signatures.

The following security services were achieved:

- ✓ Integrity
- ✓ Authentication
- ✓ Non-repudiation
- ✓ Trust Chain Validation

The experiment closely simulates real-world implementations such as:

- University digital certificate issuance
- Government e-sign portals
- Banking transaction authorization
- Corporate contract approval

Capstone Theme

“Secure University Digital Examination System”

Real-World Scenario

A university wants to:

- Ensure question papers are **authentic**
- Prevent **tampering**
- Allow exam centers to **verify signatures**
- Use **Adobe Reader** for verification

The University acts as a **Root Certificate Authority (CA)**.

The Exam Server:

- Gets certificate from University CA
- Signs question papers

The Exam Client:

- Verifies signature using University CA

This is exactly how:

- Government digital signatures work
- Banking documents are verified
- E-contracts are validated

◆ Task 1: Create University Root CA (Kali Linux)

Step 1

```
openssl genrsa -out university_ca.key 4096
```

What This Does

- Generates a **4096-bit RSA private key**
- Saves it as: `university_ca.key`

Why 4096 bits?

- Larger key = stronger security
- Root CA must be highly secure

- Used to sign other certificates

Internally

This creates:

Private Key (Secret)

This key:

- Signs certificates
- Must NEVER be shared

Student Understanding

👉 This is like creating the **University's official signing pen**.
Whoever holds this key can issue valid certificates.

Step 2

```
openssl req -x509 -new -nodes -key university_ca.key -sha256 -days 3650 -out university_ca.crt
```

What This Does

Creates a **self-signed certificate** for the University.

Breakdown:

Option	Meaning
req	Create certificate request
-x509	Create self-signed certificate
-new	New certificate
-nodes	No password on key
-key	Use this private key
-sha256	Hash algorithm
-days 3650	Valid for 10 years
-out	Output certificate file

What is university_ca.crt?

This is the **Public Certificate** of the University.

Contains:

- Public key
 - Organization name
 - Validity period
 - Digital signature (self-signed)
-

Concept for Students

Private Key → Secret signing authority

Public Certificate → Public identity card

Real World Equivalent

Like:

- Government CA
 - DigiCert
 - eMudhra
-

◆ **Task 2: Issue Certificate to Exam Server**

Now University CA will issue certificate to Exam Server.

Step 1

```
openssl genrsa -out exam_server.key 2048
```

What This Does

Creates private key for Exam Server.

Concept

Exam server now has:

Private Key (secret)

This will be used to:

- Sign question papers
-

Step 2

```
openssl req -new -key exam_server.key -out exam_server.csr
```

What is CSR?

CSR = Certificate Signing Request

It contains:

- Public key
- Organization name
- Request to CA

Analogy

Exam Server is saying:

"Dear University CA, please give me a certificate."

Step 3

```
openssl x509 -req -in exam_server.csr \  
-CA university_ca.crt \  
-CAkey university_ca.key \  
-CAcreateserial \  
-out exam_server.crt \  
-days 365 -sha256
```

What Happens Here?

University CA:

- Verifies CSR
- Signs it using its private key
- Issues certificate

Result

exam_server.crt

This certificate is:

- Signed by University CA
 - Trusted by anyone who trusts University CA
-

Student Understanding

Now we have:

File	Purpose
------	---------

exam_server.key	Private key (secret)
-----------------	----------------------

exam_server.crt	Public certificate
-----------------	--------------------

university_ca.crt	Root CA certificate
-------------------	---------------------

◆ Task 3: Convert to Adobe-Compatible Certificate

```
openssl pkcs12 -export \  
-out exam_server.p12 \  
-inkey exam_server.key \  
-in exam_server.crt
```

Why This Step?

Adobe requires certificate in **PKCS#12 (.p12)** format.

This file contains:

- Private key
 - Certificate
 - Password protection
-

Concept

This is like putting:

- Key + ID card
- Inside secure wallet
- Protected by password

◆ Task 4: Sign Question Paper using Adobe

Step 1: Import Certificate

Adobe:

Preferences → Signatures → Digital IDs → Import

Import:

exam_server.p12

What Happens Internally?

Adobe:

- Loads private key
 - Stores securely
 - Prepares for signing
-

Step 2: Sign PDF

Open:

question_paper.pdf

Click:

Tools → Certificates → Digitally Sign

What Happens Internally?

1. Adobe calculates:
 2. SHA256 hash of PDF
 3. Encrypts hash with:
 4. exam_server private key
 5. Attaches:
 - Signature
 - Certificate
 - Timestamp
-

Important Concept

Digital Signature =

Encrypted Hash

Not entire PDF encryption.

◆ Task 5: Verify at Exam Client

Step 1: Install University CA

Adobe:

Preferences → Trusted Certificates → Import

Import:

university_ca.crt

Mark:

✓ Trust for signatures

Why This Step?

Verification works on **chain of trust**.

Client checks:

1. Who signed PDF?
→ Exam Server
 2. Who issued Exam Server certificate?
→ University CA
 3. Do I trust University CA?
→ YES (because installed)
-

Internally Verification Process

Adobe:

1. Extracts signature
2. Decrypts hash using public key
3. Recalculates PDF hash
4. Compares both

If equal:

✓ Signature valid

◆ Task 6: Tampering Test

Modify PDF after signing.

🔍 What Happens?

1. Hash changes
2. Decrypted signature hash $\neq$ New hash
3. Adobe shows:

✗ Document has been altered

✗ Signature invalid

🔒 Cryptographic Flow Summary

Signing Process

PDF → Hash → Encrypt with Private Key → Signature

Verification Process

Signature → Decrypt with Public Key → Compare Hash

🏢 Real-World Correlation

This is used in:

- Government e-tenders
 - Online banking
 - Court documents
 - Passport systems
 - GST filing
 - Digital contracts
-